

Software Design for Empowering Scientists

David De Roure, *University of Southampton*

Carole Goble, *University of Manchester*

The Taverna Workbench, a scientific workflow-management system, and the myExperiment social Web site for sharing scientific experiments follow six principles for designing software for scientists.

Science is becoming increasingly digital. Scientists' tools are not just the experimental apparatus of the laboratory but are also the software apparatus they use to conduct their research, analyze data, search databases, run simulations, and record their scientific process. New scientific techniques—from DNA microarrays to sensor networks in the environment—are generating volumes of data that wouldn't get processed without software assistance.

Watch researchers at work, and you'll see a lot of computer activity as they use applications, services, and data that might be local to the laboratory or enterprise, or accessed on the Web. These new research tools and methods are evident across a broad spectrum of disciplines. Some researchers, such as bioinformaticians working with protein sequences, might conduct research entirely *in silico*. Meanwhile, chemists in the laboratory are using computers for looking up procedures, designing their experiments, conducting and recording their work, and running simulations, searches, and analyses. Climate change researchers are capturing the latest earth observation and sensor network data, and running and testing computer models. Musicologists are extracting features from recorded music as part of musical analysis. Archaeologists are exploring 3D visualizations of their digs.

Software tools are crucial for these increasingly data-intensive scientific practices, and with the right tools, a tremendous potential exists for scientific advancement. Taverna and myExperiment are two complementary tools that help scientists tackle

these challenges: Taverna provides automation of scientific data processing tasks, making them systematic and repeatable, whereas myExperiment facilitates the discovery and sharing of scientific digital objects, encouraging reuse and avoiding reinvention. These tools' comparative success owes much to a software design approach that understands the practice and culture of scientists.

Scientific Workflows

Some data processing is highly repetitive, requiring scientists to repeat processes with different data, vary parameters, and implement small changes. Clearly, automation can help speed things up and make them more repeatable. This, in turn, lets scientists concentrate on the science rather than the repetitive handling of data through multiple tools and applications. So, increasingly, scientific activities at the computer are "programmed." Some disciplines have created scripting solutions, perhaps adopting their favorite scripting languages and creating libraries to handle their discipline-specific data; others adopt workflow systems that automate data processing through a series of stages.

Taverna is one of many scientific-workflow-management systems (see the “Other Scientific Workflow Management Systems” sidebar for a brief overview of others).¹ It supports what we might call *application-level workflows*, as opposed to other systems that focus on scheduling tasks across computing resources. Emerging from the UK’s e-Science program, scientists use Taverna extensively for various life science problems: gene and protein annotation; proteomics, phylogeny, and phenotypical studies; analysis of microarray data and medical images; high-throughput screening of chemical compounds; and clinical statistical analysis. Increasingly, other disciplines are adopting Taverna, too.

Workflows lead to new scientific outcomes. For example, Paul Fisher and his colleagues have reported benefits from being able to “systematically analyse any results we obtain without the need to prematurely filter the data for human convenience,”² which enabled them to identify a candidate gene that they had previously missed. Workflows are systematic and unbiased, and they capture data analysis methodologies explicitly. Taverna’s and other workflow environments’ popularity is changing not only how scientists program their experiments—using high-level operators in terms of the problem and steps in terms of the scientific operations—but also how they propagate and communicate their experimental methods among and across communities.

Social Experiments

Scientific workflows and scripts are particularly valuable because they describe pieces of the scientific process. Providing a workflow along with experimental results helps others interpret the results, ensures that the process can be repeated, and facilitates reuse of the method. Along with this value comes a cost—writing scripts and workflows isn’t always easy. So, it’s handy to share scripts and workflows with colleagues so that they don’t have to be created from scratch. Sharing the scientific method in this way promises to be more powerful than that, however, because scientists in one domain can borrow a method from another. Figure 1 illustrates a real-world example of this.

All these benefits of sharing workflows and scripts motivated us to create the myExperiment social Web site. Using myExperiment, researchers can share experiments no matter what workflow system they use. Figure 2 shows the cycle of workflow discovery in myExperiment and editing in Taverna (for example). Although sometimes described as “Facebook for scientists,” myExperiment

Other Scientific Workflow Management Systems

In addition to the Taverna workflow system we describe in the main text, several other workflow systems have been developed to aid scientific research.¹

Triana

This open source problem-solving environment developed at Cardiff University combines an intuitive visual interface with powerful data-analysis tools. Scientists have already used Triana for a range of tasks, such as signal, text, and image processing, so it includes a large library of prewritten analysis tools and lets users easily integrate their own tools. (See www.trianacode.org.)

Kepler

This project’s overall goal is to produce an open source scientific workflow system that lets scientists design scientific workflows and execute them efficiently using emerging grid-based approaches to distributed computation. Kepler is based on the Ptolemy II system for heterogeneous, concurrent modeling and design. (See <http://kepler-project.org>.)

Pegasus

Pegasus (Planning for Execution in Grids) bridges the scientific domain and the execution environment by automatically mapping the high-level workflow descriptions onto distributed infrastructures such as the TeraGrid and the Open Science Grid. (See <http://pegasus.isi.edu>.)

Reference

1. I.J. Taylor et al., eds., *Workflows for e-Science: Scientific Workflows for Grids*, Springer, 2007.

Recycle, Reuse, Repurpose Workflows and Know-how

- Paul writes workflows for identifying biological pathways implicated in resistance to Trypanosomiasis (caused by the parasites *Trypanosoma congolense* and *Trypanosoma vivax*) in cattle and mice (see [2])
- Paul meets Jo. Jo is investigating *Trichuris muris*, another parasite but this time in mouse. She is using the same kind of applications and services, and some different data sets
- Jo reuses one of Paul’s workflows **without change**.
- Analysis of the resulting data by Jo identifies some biological pathways involved in sex dependence in the mouse model, believed to be involved in the ability of mice to expel the parasite. It takes two days for the analysis.
- Previously a manual **two year study** by Jo had been unable to do this.

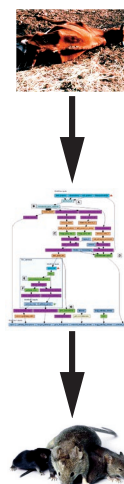


Figure 1. Scientific workflow example. Here, a workflow is successfully reused to achieve a new scientific outcome. (With thanks to Paul Fisher and Joanne Pennock, the University of Manchester.)

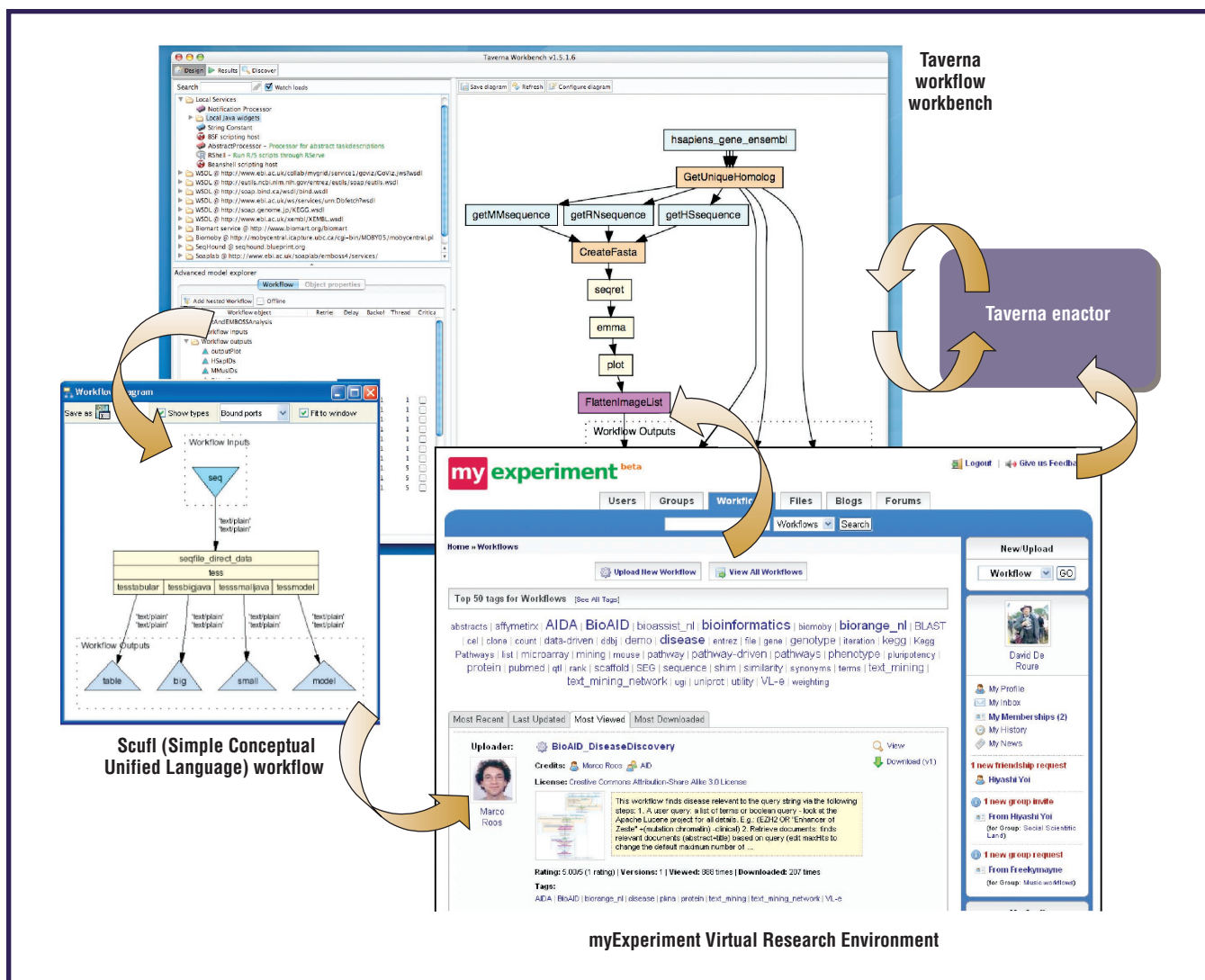


Figure 2. Workflow discovery in myExperiment. A scientist discovers a workflow in myExperiment, runs it and edits it in Taverna, and uploads it back to myExperiment.

is different from sites for sharing pictures, slides, and so on because it focuses on scientists' special requirements, such as the need to attribute work, control visibility, handle licensing, and work with distributed data collections. myExperiment couples decoupled communities and brings community intelligence to workflow systems.

Take workflow reliability, for instance: a workflow's success depends on all the services being available and working as expected, which is a challenge. Many services that are linked as steps in a workflow are third party, autonomous, and changeable, so workflows tend to decay. By sharing workflows within a community, we can achieve community recommendations and workflow maintenance. By tracking workflow use, myExperiment can recommend popular and reliable workflows and services. It also lets us exchange scripts and workflows for multiple tools, broadening the scientist's toolkit.

The Software: Superclients and the Web

Taverna is a distinctive workflow system because it's a *superclient*, meaning anyone can download it and run it on his or her desktop or laptop PC without extra installations or system administration. You can use it to plumb together any services you can access on and from your PC, whether they're in the enterprise or on the Web, by running a Taverna workflow. The idea is that using Taverna should be as simple as using a Web browser, except that it runs workflows instead of displaying Web pages.

Taverna has three main parts. The Taverna enactor is the engine that takes a Taverna workflow and executes it, using the data the user provides, over the services described in the workflow. In the enactor's early form, it did simple data staging from service to service; it's evolved to support streaming, runtime determination of services from

service groups, and numerous extension points for developers.

The Taverna Workbench provides a graphical editor for workflows and, significantly, a means for users to choose services. The availability of services for a particular domain is what makes Taverna easy to use in that domain. Adding new services is easy, supported by a scavenging tool and service catalogues. Finally, the Taverna language is a simple dataflow language called the Simple Conceptual Unified Language (Scufl) that has implicit iteration constructs, manifested graphically in the Taverna editor and encoded in XML. Scufl is a declarative language with its semantic roots in functional programming, and it can interoperate with several other scientific workflow tools. It's quite different from languages such as WS-BPEL (Web Services Business Process Execution Language), which support a control-flow paradigm.

Meanwhile, myExperiment is a very different piece of software—a Web-based application, built on the Ruby on Rails (RoR) platform, that provides a modern Web 2.0 look and feel, social features, and lean APIs. Rather than just a single site, such as Facebook or YouTube, myExperiment is also a software package that you can install independently in a laboratory. It supports content exchange between Web applications, different myExperiment installations, and digital repositories. It's also designed to be extensible, so that it deals not in workflows or scripts per se but in scientific objects. This enables sharing documents, presentations, service descriptions, notes, ontologies, plans, and so forth. More generally, myExperiment deals in experiments; you can use it to glue together heterogeneous collections such as distributed experimental data or, for example, packs of workflows. We call these collections *encapsulated myExperiment objects*.

Software for Science

Taverna has been adopted extensively, topping 55,000 downloads from Sourceforge to date and averaging more than 40 a day since 2006. myExperiment went from conception to open beta in nine months, with the code base delivered by a core team of two developers. Now at 12 months, it has more than 1,300 users, despite limited advertising and being such a specialist tool.

Although adoption of Taverna has been broad, this isn't always the case—we've developed systems in the past that were fine examples of well-designed software but were disdained or neglected by their intended users. Scientists have challenging and changeable applications that they might really

understand but that are hard to communicate or stabilize. They're concerned with getting scientific results rapidly to compete with their peers and develop their reputations. Scientists want reliability and confidence in their software but, paradoxically, are less concerned with generic solutions that they or others can replicate among their communities and that evolve systematically. On the other hand, these very things motivate software engineers, who want to build the best solution and one that many other users will adopt, but who aren't the specialists who will ultimately use the software. Sometimes, computer scientists are in the mix, too, and they must compete with their academic peers by exploring interesting technologies and innovative methods that could well be at odds with the scientists' and software engineers' needs.

Six Principles of Designing for Adoption

Scientists, to be successful, must be fundamentally selfish—Mike Ashburner, a Cambridge geneticist, once said that “Scientists would rather share their toothbrush than their data.” You must recognize that they adopt software on the basis of what it will do for them. So, what counts the most? It's not clever software or innovative technologies but rather content and relevance. When we developed Taverna, we established a set of design principles and a working practice with our scientists, which myExperiment evolved further.³

1. Fit In, Don't Force Change

We made no obligation for service providers to change their services to fit into Taverna. Instead, we rapidly assembled content—more than 3,500 service operations covering all the major service providers in the community (the bioinformatics community routinely uses more than 900 databases⁴). Our early users were prepared to tolerate less-than-perfect interfaces because their favorite service was available. Users don't have to change how they work to use the system—they simply download Taverna and can use workflows immediately, viewing their data with the same applications as before. Its functionality sits comfortably with their existing environment and practice, and barriers to adoption are kept low at the expense of, for example, elegant-type systems.

In myExperiment, our motto is “bring myExperiment to the user,” rather than force the user to come to myExperiment. So, for scientists who are already using Web sites and wikis in their work, we make it easy to bring myExperiment functionality through their existing interfaces.

We've developed systems in the past that were fine examples of well-designed software but were disdained or neglected by their intended users.

**Taverna
has immediate
benefit because
it automates
repetitive,
time-consuming
tasks.**

For those who don't, we made the Web interface as familiar as possible.

2. Jam Today and More Jam Tomorrow

This is about incentives. Incremental development and incremental content give scientists an immediate return on the time and effort they invest. So, we must match the *activation energy* users require to adopt a feature with the reward they gain. We must get core capability and quick wins out promptly. We built our user communities the way we built the software: scientists and developers journeying together. Telling our scientists to commit time and effort to something that they wouldn't get any benefit from along the way would have been pointless.

Taverna has immediate benefit because it automates repetitive, time-consuming tasks and promises greater value in the future as more workflows become available and as users become workflow creators (if they wish). myExperiment gives an immediate return by providing examples of workflows others have written and the promise of more workflows and community help.

3. Just in Time and Just Enough

Coupled with the previous point, we learned not to try to be too smart (always a temptation for computer scientists). Being "better, not perfect" lets us respond to users' immediate needs rather than going away and building a complete solution—which would be more resource intensive and, in our experience, quite possibly wrong. It's better to solve the problems users already know they have than to make them wait for solutions to problems they don't yet know they have and perhaps never will.

In Taverna, for example, we worked on fancy knowledge-based descriptive techniques for services to automate workflow composition. It turned out that users didn't want this functionality at all. They wanted quick ways to find relevant services, and then they wanted help in building the workflows themselves. Building software to power a Web site such as myExperiment is different from building workflow system software—for one thing, we found it easier to be incremental, involve a distributed and disparate user community in the design process, capture their input, and be agile in response.

4. Act Locally, Think Globally

Instead of setting out to build a universal workflow system or a universal social-networking site, we targeted a community we already knew. Moreover, we picked a few close "friends and family"—that is, local pioneers who are stereotypical exam-

ples of a certain class of scientists with a certain class of problems—and built our system for them, working closely with a few key service providers as well. Ultimately, when our local scientists were happy, so were those we'd never worked with before. So, we delivered Taverna with sets of services that met target user communities' specific needs and made it easy to extend to others. Meeting local needs encourages adoption; making the software extensible broadens it. Our experience suggests that customization outvotes genericity, extensibility outvotes comprehensivity, and scruffy and flexible outvotes smart but rigid.

In myExperiment, we never made assumptions about what users wanted. Because we were limited to a core team of two developers, it was essential to impose a simple mantra: only implement features that users have asked for. Our failures occurred when we tried to second-guess a solution for which there was no clear user problem statement. As Don Wells said, "Don't solve a problem before you get to it."⁵

5. Let Users Add Value

Extensibility and customization are crucial to adoption because they empower users. Users best understand the visualization tools they want to use or the services they want to incorporate. Many are capable developers or have developers working with them. Taverna lets users and developers enhance the system by creating content, integrating services, and developing software. They can extend it through new services and plug-ins and can share these extensions so that Taverna's value accumulates. We don't need to create the extensions ourselves but rather provide support and training for others to do so.

The small team associated with myExperiment means that maximal reuse and reusability is a necessity. We aimed to use third-party code and services as much as possible and to make our services available through a simple Representational State Transfer (RESTful) API. We published this API as early as possible to let others start development and build functionality mashups. Within hours of releasing the API, we had more people developing outside the team than within it. By providing a network of cooperating data services with simple interfaces that make working with content easy, we're both providing and reusing services. myExperiment also supports lightweight programming models for ease of integration in loosely coupled systems.

6. Design for Network Effects

Our users aren't just heroic e-scientists harnessing computing resources to tackle major scientific

breakthroughs; we also work with numerous researchers conducting routine processes on a daily basis, service providers who supply the resources they use, and workflow developers willing to share their know-how. Harnessing this long tail enables network effects and provides community intelligence.

In myExperiment, finding workflows must be easy, and sharing and adding them and other scientific assets to the pool must be useful and straightforward. Usage statistics for workflows and services provide a basis for recommendations, as well as the more explicit mechanisms of reviewing and tagging favorites.

These principles interconnect, of course, and are predicated on well-designed, loosely coupled modular software. Principles 2 and 3 led us to adopt a perpetual beta software development methodology. Taverna uses a plug-in and code manager that manages third-party-supplied extensions and provides automatic updates. We develop and test the myExperiment codebase continuously and make weekly updates to the live service. Both projects are just as much—or perhaps even more—about content as software. We were reminded of this when we went to open beta with myExperiment; the first bug report that came in was about a spelling error in a user-provided workflow description. Users come to the site for workflows and, as with Taverna, we’ve been careful to bootstrap the system so that the users have “jam today.”

Building Trust between Users and Developers

These six design principles depend on a good relationship between users and developers. Taverna began as an e-Science pilot project in 2001; one-third of the team that built the early prototypes were bioinformaticians. Now, a six-person software engineering team, together with a three-person user outreach team (also bioinformaticians) delivers and supports the software as part of the Open Middleware Infrastructure Institute UK (OMII-UK) and in close collaboration with its user communities.

Throughout myExperiment, we’ve operated with two core developers, with a larger team around them providing occasional support, including part-time liaison staff for specific research communities.

We’ve achieved this collaboration in both projects through our six principles of user engagement. Figure 3 shows these in relation to our six design principles.

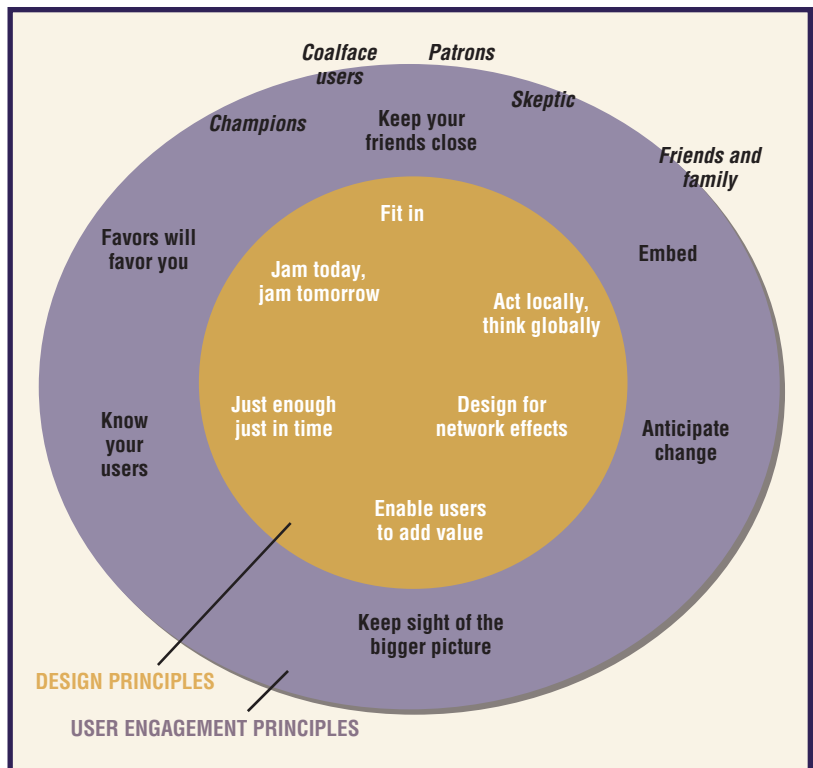


Figure 3. Principles of software design for adoption and user engagement. Achieving the design principles (inner circle) depends crucially on the relationship between the software providers and the users, achieved through the user engagement principles (outer circle).

1. Keep Your Friends Close

In addition to community development with local pioneers and external early adopters, OMII-UK operates a scheme of beta testers and product/area liaisons (PALs) who don’t work directly for OMII-UK but are the eyes and ears in the communities they do work in. Taverna’s PALs are critical to identifying user needs and developing the community. By choosing these individuals, we recognize their worth to both OMII-UK and to their community; all parties benefit and so does the design and adoption of the software.

For myExperiment, we built user advocacy into its management plan. User workshops (dubbed *parties*) are combined with *community champions*. Our friends-and-family users (including sceptics!) and the broader communities are involved at all times—through everyday use of the Web site, consultations about mockup page designs, and user evaluation sessions.

Our advocates are all our software’s day-to-day users—mostly postdoctoral scientists and postgraduates. In academia, we can achieve software adoption through these users rather than by targeting more senior scientists. However, if we want their (and community) support, they must have sympathetic patrons such as professors and laboratory managers. A more controversial viewpoint is to pick good scientists who are seeking a competitive edge. Those who want to overtake

**Letting users
add value
gives them
ownership of
the software—
another plus
encouraging
adoption.**

their peers will look to new tools to help them and will be more willing to try them out.

2. Embed

We embed developers with users and users with developers, getting them to sit side by side for long periods. No amount of requirements documentation or design meetings can compensate for day-to-day contact. We started myExperiment by embedding the developers in an end-user laboratory so they could gain first-hand experience with the work environment, particularly with respect to sharing, communication, and understanding nonfunctional requirements.

3. Look at the Bigger Picture

Developers and computer researchers tend to get hung up on some point that users won't worry about in the overall scheme of things. Our software isn't the only software our users use, and they're using it to do only the science they really want to. Day-to-day, they use complex tools that they're familiar and comfortable with and metaphors that seem peculiar to those on the outside but are standard in their discipline. Mimic these tools and link with them because they're part of the overall process. Consider the whole life cycle—we can accelerate science not just by accelerating experiments but also by reducing the time it takes to devise an experiment.

4. Favors Will Be in Your Favor

Building trust is a two-way activity requiring compromise and favors. In Taverna's early days, we often wrote specific code to help our pioneering scientists get their scientific results. For every pioneer, we "sacrificed" one developer in support. You could argue that this effort was a waste, but of course it wasn't. The developers learned about the problem, and the users felt we were on their side. The developers got "jam tomorrow," too, receiving added, patient attention from scientists when it came to evaluating prototypes and designs. Letting users add value gives them ownership of the software—another plus encouraging adoption. Today, we still develop—or help partners develop—bespoke code that leads to a general release (design principle 4). In myExperiment, we prioritize tasks and allocate development efforts on the basis of the return in adoption.

5. Know Your Users

Rarely is there one kind of user. For example, for our software, we have bioinformaticians and biologists (two distinct user groups), domain-

expert developers producing applications, service developers, service providers, and system administrators. All have different needs and fears. Taverna is designed for expert bioinformaticians, not biologists.

Also, know your users' backgrounds and experiences. The bioinformaticians who use Taverna are young postdocs and postgraduates familiar with online gaming, instant messaging, file sharing, social networking, and Internet shopping sites. So, basing myExperiment on this experience and others was natural. Searching for workflows should be like shopping on Amazon or searching with Google; creating work groups should be like creating groups on Facebook. Familiar interfaces mean no training.

6. Expect and Anticipate Change

User needs will change. Success will mean that scientific practice will change, as will expectations. Scientists who were happy with a slow response or minimal data-management support will now need production-level database support and reliable, rapid delivery. You will need to find new pioneers, and new user classes will emerge.

When we started out, we replicated current scientific practice. The focus was on interoperating services, making services accessible, and workflow creation.⁶ Two years later, we were inventing new practice. Workflows had to integrate data, not just link services, discovery had become an issue with so many services, and workflow editing and reuse were as important as creation from scratch. Now we have a workflow repository that is used and curated by the community and is itself being accessed by scripts and workflows. The software changes the science, which changes the software. The challenge for developers is predicting and anticipating these trends within the six design principles.

Reflections

We based our six design principles on several years of experience in the e-Science program, observing both success and failure. In this article, we've looked at the design of two quite different pieces of software—the superclient and the Web application. Although they're both in the workflow domain, we believe we can apply our design principles more broadly. They actually cover software design's social aspects, which is significant in the collaborative context of e-science. They might extend also to systems-level science, which "integrates not only different disciplines but also, typically, software systems, data, computing resources, and people."⁷ This broader applicability

is exemplified by our chemistry colleagues who are “blogging the lab” and have taken a sympathetic design approach.⁸

Being able to realize our principles in engineering practice brings trade-offs. For example, using RoR in myExperiment imposed something of a straitjacket that made it difficult for us to act locally with multiple user interface models while thinking globally in the back end. However, having a programming model that lets us spend more time with users and less time writing code has been hugely effective. We anticipate extensive use of Web 2.0 functionality mashups for scientists. Considerable consistency exists between our design and engagement principles, and Web 2.0 design patterns.

In this article, we focused on our principles rather than their implementation. Managing myExperiment has much in common with Extreme Programming (XP), given that we implemented a core team across two sites with one-week iterations, had onsite customers at both, and performed continuous integration. Although others have discussed XP for scientific research,⁹ we’ve explored a rather different scenario—decoupled communities of independent and autonomous scientists doing everyday research, which is the emerging world of open science. We found it natural to be agile and responsive with a Web application.

We’re both delivering software to scientists and empowering them (and their computing experts) to participate in software creation—software is the power behind the scientists, and scientists are the power behind the software. We believe this empowerment is essential. Being prescriptive won’t work because scientists must be as comfortable using their software apparatus as they are any other scientific tool. It’s not so much about rolling out software as rolling in users, and it can happen only if software designers are willing to cross the line and take a user-centric view of the software design process. ☞

Acknowledgments

The Taverna team, past and present, is extensive: special thanks to the lead architect, Tom Oinn, and to Hannah Tipney, our first bio-pioneer. Thanks to all our developers, product/area liaisons, advocates, and myExperiment “friends and family,” including Jiten Bhagat, Andy Brass, Simon Coles, Don Cruickshank, Paul Fisher, Jeremy Frey, Antoon Goderis, Duncan

About the Authors



David De Roure is a full professor in the School of Electronics and Computer Science at the University of Southampton, where he was a founding member of the Intelligence, Agents, Multimedia Group and is responsible for e-science. He’s the director of the myExperiment project and has UK and international roles in developing interdisciplinary e-research activities. De Roure received his PhD in computer science from the University of Southampton. He’s a fellow of the British Computer Society. Contact him at dder@ecs.soton.ac.uk.

Carole Goble is a full professor in the School of Computer Science at the University of Manchester, where she coleads the Information Management Group. She has worked closely with life scientists for many years and is the director of the myGrid project, the largest UK e-science pilot project, which has produced the widely used Taverna Workflow Workbench open source software. Goble is a fellow of the British Computer Society. Contact her at carole.goble@manchester.ac.uk.



Hull, Douglas Kell, Matt Lee, Bertram Ludäscher, Danus Michaelides, Stuart Owen, Rob Procter, Marco Roos, Stian Soiland, Robert Stevens, Ian Taylor, David Withers, and Katy Wolstencroft.

References

1. T. Oinn et al., “Taverna: Lessons in Creating a Workflow Environment for the Life Sciences,” *Concurrency and Computation: Practice and Experience*, vol. 18, no. 10, 2006, pp. 1067–1100.
2. P. Fisher et al., “A Systematic Strategy for Large-Scale Analysis of Genotype–Phenotype Correlations: Identification of Candidate Genes Involved in African Trypanosomiasis,” *Nucleic Acids Research*, vol. 35, no. 16, 2007, pp. 5625–5633.
3. D. De Roure, C.A. Goble, and R. Stevens, “Designing the myExperiment Virtual Research Environment for the Social Sharing of Workflows,” *Proc. 3rd IEEE Int’l Conf. E-science and Grid Computing (E-science 07)*, IEEE CS Press, 2007, pp. 603–610.
4. M.Y. Galperin, “The Molecular Biology Database Collection: 2007 Update,” *Nucleic Acids Research*, vol. 35, 12 Jan. 2007, pp. D3–4.
5. A.I. Antón and D. Wells, “Point/Counterpoint: Don’t Solve a Problem Before You Get to It/Successful Software Projects Need Requirements Planning,” *IEEE Software*, vol. 20, no. 3, 2003, pp. 44–47.
6. R. Stevens et al., “Exploring Williams-Beuren Syndrome Using myGrid in Bioinformatics,” *Bioinformatics*, Aug. 2004, pp. 303–310.
7. I. Foster and C. Kesselman, “Scaling System-Level Science: Scientific Exploration and IT Implications,” *Computer*, vol. 39, no. 11, 2006, pp. 31–39.
8. C. Neylon, “Workshop on Blog Based Notebooks,” *Science in the Open: An Openwetware Blog on the Challenges of Open and Connected Science*, 2008; <http://blog.openwetware.org/scienceintheopen/2008/02/29>.
9. W.A. Wood and W.L. Kleb, “Exploring XP for Scientific Research,” *IEEE Software*, vol. 20, no. 3, 2003, pp. 30–36.

For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.